

Abstract

This project investigates the computational trade-off in real-time computer vision systems between processing speed and detection accuracy. Traditional deep learning models often perform well in object detection but are far from ideal for continuous monitoring due to their high computational requirements. To address this, we developed a hybrid architecture that integrates pre-trained Haar Cascade classifiers from the OpenCV library with the YOLO11 (You Only Look Once) object detection algorithm. Our approach utilises a hierarchical decision logic in which the Haar Cascade serves as a rapid, low-resource first layer to detect faces, significantly reducing computational overhead by triggering the heavier YOLO11 model only when no face is present. The YOLO11 model is pre-trained on the Common Objects in Context (COCO) dataset, which contains over 330,000 images and 80 categories, enabling the detection of common everyday objects. The final system demonstrates effective real-time performance, optimising speed and proving the efficiency of the hybrid architecture.

Keywords: Computer Vision, Face Detection, Object Detection, Haar Cascades, YOLO11, Real-time Processing.

Table of Contents

Introduction & Background.....	5
1.1 Context & Motivation.....	5
1.2 Problem Statement.....	5
1.3 Literature Review.....	5
Theory & Methodology.....	6
2.1 Theoretical Foundation.....	6
2.2 Proposed Method.....	7
2.3 Prioritisation and Decision Logic.....	8
2.4 Visual Output.....	9
Dataset & Implementation.....	10
3.1 Datasets and Model Selection.....	10
3.2 Preprocessing Steps.....	12
3.3 Implementation Details & Evaluation Metrics.....	13
Results & Analysis.....	15
4.1 Face Detection Results.....	15
4.2 Face Detection Sample Results.....	17
4.3 Object Detection Results.....	18
Conclusion & Future Work.....	20
Contributions of Team Members.....	22
References.....	23

List of Tables

Table 1: The Dependence of Algorithms.....	6
Table 2: Prioritisation and Decision Logic.....	8
Table 3: Preprocessing Steps Comparison.....	13
Table 4: Team Responsibilities.....	22

List of Figures

Figure 1: Conceptual Flow of the System.....	10
Figure 2: YOLO11 Compared to Other Versions.....	11
Figure 3: Face Detection Results.....	16
Figure 4: Face Detection Errors.....	16
Figure 5: Confusion Matrix.....	18
Figure 6: Object Detection Result.....	19
Figure 7: Object Detection Error.....	19

Introduction & Background

1.1 Context & Motivation

To evaluate automated judgments and create data from visual aspects, today's digital rely significantly on computer vision [1]. Whether in the form of a camera for security, a mobile application, or a smart device sensing environmental cues, visual detection systems have become essential, capable of identifying people or recognizing objects. Detecting whether a scene contains a human face or an inanimate object is especially useful in many real-world contexts. To design a basic yet intelligent vision model, the developed program in this project implements a sequential system that first detects human faces, and when no face is found, automatically performs object detection to identify everyday items.

The goal is to create a real-time solution that operates smoothly even on devices lacking powerful graphics hardware. The project is applicable in areas such as home item monitoring, classroom demonstrations, and entry-level automation. Moreover, this dual-function strategy enhances practicality and efficiency.

1.2 Problem Statement

Most modern systems treat face and object recognition as independent tasks. According to Edozie et al. [2], in real-world contexts, a fundamental constraint is the lack of a unified framework that first detects the presence of a face and then switches to object recognition as needed, or can perform both when required. Without this logical sequence, systems often fail to capture relevant information and may waste unnecessary processing resources.

In a real-time automated environment, this project aims to develop a compact and efficient sequential model capable of first identifying human faces and then detecting general objects when no faces are present.

1.3 Literature Review

With the traditional techniques of image processing, the face detection research began. The influential technique of Viola-Jones allowed for quick localization of faces in live applications, which introduced Haar-based detection [3]. Even with the advent of deep

learning, Haar-based detection remains popular due to its computational efficiency and suitability for real-time use [4].

According to Ali and Zhang [5], with the You Only Look Once (YOLO) family being notable for its single-stage detection approach, object detection research has shifted almost entirely toward deep learning. YOLO increases responsiveness and reduces computational power requirements by processing an entire image at once, both of which are crucial factors for real-time recognition tasks [6].

With its improved backbone design, according to He et al. [7], YOLO11 enhances the architecture through anchor-free detection modes and advanced optimization techniques. Studies comparing various approaches suggest that classical models can complement deep-learning frameworks by performing rapid filtering before applying more complex models [8]. This supports the use of Haar Cascades for fast face detection, followed by YOLO11 for object identification when no face is visible.

Theory & Methodology

2.1 Theoretical Foundation

The system functions sequentially, combining two recognition modules that operate in a single integrated pipeline.

1. Haar-Based Face Detection

Haar Cascades rely on simple visual patterns composed of light and dark regions. These features are scanned across the image to detect structures resembling human facial characteristics such as the eyes, nose, and chin margins. The method is computationally efficient and well-suited for real-time applications.

Table 1: The Dependence of Algorithms

Algorithm	Depends on
Haar-Based Face Detection	1. Integral Images: for faster feature calculation. 2. Cascaded Classifiers: to reject non-face regions and minimize computation.

	3. Adaptive Boosting (AdaBoost): to select the most significant Haar features and combine weak classifiers into a strong face detector.
--	--

Haar-based detection runs first to serve as a “gatekeeper” in the system. Due to its computational efficiency and the qualities mentioned in Table 1, it reduces the processing load on the deeper model while maintaining sufficient speed for real-time use [9].

2. YOLO11 Object Detection

To partition an image into grids, YOLO11 uses a deep neural network that predicts bounding boxes and class scores in a single pass. Since it does not require a separate region-proposal stage, the model efficiently extracts multi-scale features and performs end-to-end detection. YOLO11 optimizes multiple loss functions simultaneously, including bounding box accuracy, object confidence, and class probability, while using convolutional layers for hierarchical feature extraction [10].

In multi-object situations, YOLO models are well known for their speed and reliability, making them ideal for detecting everyday items such as bottles, chairs, and phones.

2.2 Proposed Method

The system uses a pipeline of structured criteria:

1. Processing of Input

- a. Each frame is captured from the webcam in real time and preprocessed for analysis.
- b. The Haar Cascade classifier receives a grayscale version of the frame for faster computation.
- c. The YOLO11 receives the same frame in BGR format from OpenCV, which is automatically converted to the RGB format, normalized, and ready for inference.

2. Initial Face Detection

The system first attempts to locate facial features using the Haar Cascade classifier.

1. When a face is detected, a red bounding box is drawn around it, and the message “Face Detected” is displayed on the screen.
2. According to Du et al. [11], performing the initial face detection stage reduces unnecessary computation by preventing the heavier object detection model from running when it is not needed, thereby improving overall efficiency.

3. Object Recognition Stage

- a. If no faces are detected, the system automatically triggers the YOLO11 object detection model.
- b. When an object is detected, a green bounding box, object name, and confidence score are shown around it, and an “Object Detection Active” message is displayed on the screen.
- c. Based on the patterns learned from the COCO training dataset, YOLO11 classifies up to 80 object classes and labels multiple objects within the frame (e.g., bottle, phone, dining table).

2.3 Prioritisation and Decision Logic

Table 2: Prioritisation and Decision Logic

Prioritisation and Decision Logic	Description
The system follows a simple rule of prioritisation:	a. Face detection takes priority in the sequence. b. If no face is detected, the system automatically switches to object detection and displays the bounding box around detected objects with their names and confidence scores.
Decision Logic & Prioritisation	● Avoids overlapping or conflicting outputs between the two detection stages. ● Reduces computational load by activating YOLO11 model only when necessary.

	• Decreases the overall processing time while maintaining efficiency. • In computer vision research, according to Haruna et al. [12], such hybrid approaches align with the principles of efficient and adaptive model design.
--	---

2.4 Visual Output

Finally, to provide a clear visual representation of the final processed frame, the system displays text labels and bounding boxes over the detected regions. During face detection, red rectangles are drawn around identified faces with a “Face Detected” label. When no faces are present, the object detection model activates to detect and label common objects, drawing green rectangles with corresponding names and confidence values as well as an “Object Detection Active” label.

Conceptual Flow of the System

As illustrated in Figure 1, the system follows a sequential pipeline: a lightweight Haar-based model first checks for faces, and only when no faces are detected does the deeper YOLO11 model engage for object recognition. This design ensures responsiveness, reduces computation time, and maintains real-time performance even on systems with limited hardware capabilities.

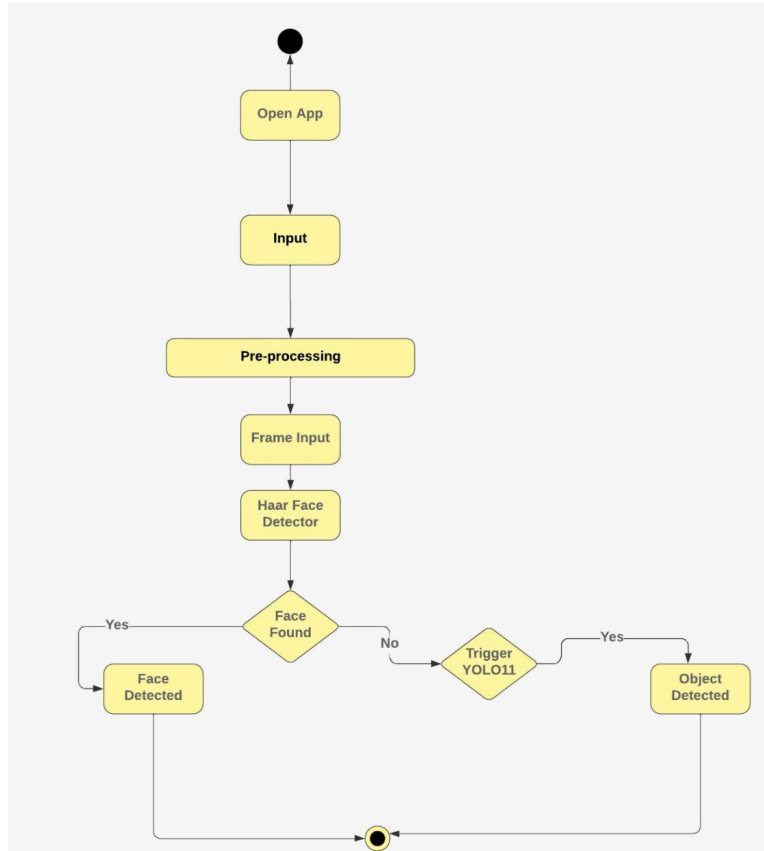


Figure 1: Conceptual Flow of the System

Dataset & Implementation

3.1 Datasets and Model Selection

Our system utilises two models, each serving a specific purpose in our sequential design:

- Face detection: For the initial stage, we utilised the pre-trained Haar Cascade classifiers provided by the OpenCV library. These models use a cascade function trained on positive (face) and negative (background) images to identify morphological features. We selected Haar Cascades specifically for their computational efficiency. They are the first layer and act rapidly as a gatekeeper, processing frames in

milliseconds to determine if a human is present before it considers calling the heavier deep learning model.

- Object detection: When the gatekeeper model determines that object detection is required, we employed YOLO11, the latest version in the “You Only Look Once” family by Ultralytics for that purpose. We selected this version specifically over previous versions (like the commonly used YOLOv5 or v8) because it offers a more efficient backbone, resulting in more accurate results and a lower runtime.

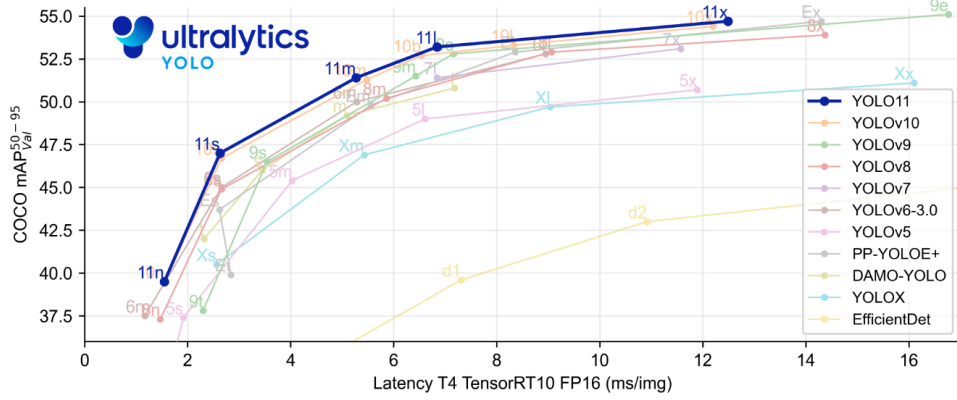


Figure 2: YOLO11 Compared to Other Versions

As shown in Figure 2 [13], YOLO11 shows significant improvement in computational latency and detection accuracy compared to all the previous versions. This indicates that for any given latency (x-axis), YOLO11 achieves higher Mean Average Precision (mAP). For example, the YOLO11n (nano) model achieves a mAP of nearly 39.5 while maintaining a latency of roughly 1.5ms on T4 hardware. This ensures our system remains real-time even when switching from the lightweight Haar Cascade to the YOLO model.

Our project goal is to optimize resources, and YOLO11 uses 22% fewer parameters, which makes it achieve higher accuracy while consuming less RAM and storage. This makes it the ideal choice for the sequential logic of our project.

The model is pre-trained on the COCO (Common Objects in Context) dataset, enabling it to recognise 80 different classes of everyday objects (e.g., phone, bottle, sofa, and chair) with high confidence on the first pass. It not only excels in detecting objects of different sizes but can even successfully identify partially hidden objects without requiring the user to wait longer for results.

While several object detection methods exist, we chose YOLO because its design fits perfectly with real time hybrid systems. Unlike two stage models such as Faster R-CNN or Mask R-CNN (Region Convolutional Neural Network), which first search for possible object areas and then classify them, YOLO completes both steps at once. This end to end design aligns with our system's need for immediate responses rather than batch based accuracy optimization.

Compared to other single stage detectors like Single Shot Multibox Detector (SSD) and RetinaNet, YOLO uses a similar one step approach. However, its newer versions such as YOLOX and YOLOv8 use improved anchor free designs that let the model learn object positions directly rather than depending on present box shapes. This makes training datasets simpler and more flexible. While SSD and RetinaNet are also strong options, YOLO's balance of speed, accuracy, and easy setup makes it one of the best choices for real time systems.

We also considered transformer based models such as Detection Transformer (DETR). These models work differently from Convolutional Neural Network (CNN) based detectors like YOLO. Instead of scanning parts of an image through layers of filters, transformer models divide the image into small patches and analyze how these patches relate to one another. They use a process called self attention, which helps the model focus on the most relevant areas of the image and understand how multiple objects connect within a single scene. This allows transformers to recognize complex spatial relationships and capture the overall context of the image. However, these benefits come with higher requirements for memory, processing power, and training time. Transformer based models are more suitable for large scale or cloud based systems rather than lightweight or real time applications.

YOLO's straightforward structure, quick setup, and ability to run efficiently on both central processing units (CPUs) and graphic processing units (GPUs) make it an excellent match for our Haar Cascade gatekeeper model. Together, they keep the system fast, efficient, and easy to use without needing special hardware.

3.2 Preprocessing Steps

To ensure the optimal performance of the lightweight and deep learning model, the input video frames require preprocessing measures first. Table 4 highlights the different kinds of preparation required for every model.

Table 3: Preprocessing Steps Comparison

Preprocessing Step	Haar Cascade (Face Detection)	YOLO11 (Object Detection)
Color Space	Grayscale (1 channel)	RGB (3 channels)
Resizing Technique	Standard downscaling	Letterbox resizing
Normalization	None	Scale to 0.0 - 1.0
Input Structure	2D array (Height, Width)	4D Tensor: (Batch, Channels, Height, Width)

For Haar Cascade, the input video needs to be converted to grayscale before any detection is performed. This is because Haar Cascade classifiers operate by calculating the difference in pixel intensities between adjacent rectangular regions. This model depends on structural patterns rather than color, so converting to grayscale is a necessary step, which also helps improve processing speed [14].

For YOLO11, the frame undergoes “letterbox” resizing. This means images are resized while maintaining aspect ratio, preventing any distortion. Additionally, pixel values are normalized (scaled from 0 - 255 to 0.0 - 1.0) to stabilize gradients [15].

3.3 Implementation Details & Evaluation Metrics

The main concept of our implementation [16] of the hierarchical approach is to minimize computational costs. As Figure 1 illustrates, the system follows a conditional execution flow:

- 1. Initial filtering:** Every frame is first analyzed by the lightweight Haar Cascade classifier.
- 2. Conditional branching:**
 - a. If a face is detected: The system marks the frame as “Face Detected,” draws a bounding box, and immediately skips the deep learning stage.

- b. If no face is detected: The YOLO11 model is triggered for deeper analysis to perform a scan for general object detection.

To accurately evaluate the performance of our proposed model, we will follow the standard computer vision metrics:

- **Frames Per Second (FPS):** This metric reflects how efficiently each frame is processed per second during live execution. FPS Delta was analyzed conceptually by comparing the difference in processing speed when the system operates in Haar only or YOLO only. High FPS during the face detection stage confirms that our lightweight initial layer successfully reduces computational load [17].
- **Intersection over Union (IoU):** This metric measures the localization accuracy by calculating the ratio of the area of overlap between the predicted bounding box and the true box area to the area of their union.

$$\text{IoU} = \frac{\text{Area of Overlap (Intersection)}}{\text{Area of Union}}$$

A prediction is considered a “true positive” only if the IoU exceeds a threshold of 0.5. This ensures the system can accurately determine the exact location of objects in the frame.

- **Mean Average Precision (mAP):** It calculates the average precision across all 80 classes of categories from the COCO dataset at a specific IoU threshold of 0.5. We utilize mAP to validate the selection of YOLO11 rather than older models used for smaller projects [18].

To prioritise overall model speed, our model does not calculate these statistical metrics in real time, as doing so would require additional computational overhead. Instead, the values presented in this report were derived by estimation through observing the YOLO11 runtime performance during testing.

In addition to these metrics, the face detection stage was also evaluated as a binary classification problem. Since the Haar Cascade acts as a face presence detector, each frame can be classified as either “Face Detected” or “No Face Detected”. The following metrics were used to assess the reliability of the face detection stage:

- **Confusion Matrix:** It summarizes the model’s predictions using four categories.

- True Positive (TP): A face is present and correctly detected.
- False Positive (FP): The model detects a face where there is none.
- True Negative (TN): The model correctly identifies that no face is present.
- False Negative (FN): A face is present but not detected by the model.
- **Precision:** Indicates how many of the detected faces are actual faces.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall:** Measures how many real faces in the dataset were successfully detected.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **Accuracy:** Represents the overall correctness of the model across both face and non-face classifications.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- **F1-Score:** Provides a balanced measure of both precision and recall.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Results & Analysis

4.1 Face Detection Results

Error-free, real-time facial detection is considered essential for most applications, such as surveillance, human-computer interaction, as well as others. In this regard, this section showcases the effectiveness of our Haar Cascade facial detection system by testing its applicability within real-world applications. We will test its capabilities based on three main aspects: accuracy, complexity and speed.



Figure 3: Face Detection Results

Across a variety of input photos, the Haar Cascade model has strong face detection capabilities. Visual examination shows that even in complex images with cluttered backgrounds, the model still manages to correctly recognise faces. In Figure 3, the second result detects a face that is partially covered by a mask. This implies that the model is not unreasonably sensitive to slight differences in looks and has mastered the ability to extract prominent facial traits. Additionally, the faces are securely enclosed by the bounding boxes produced by the model, indicating accurate localization.

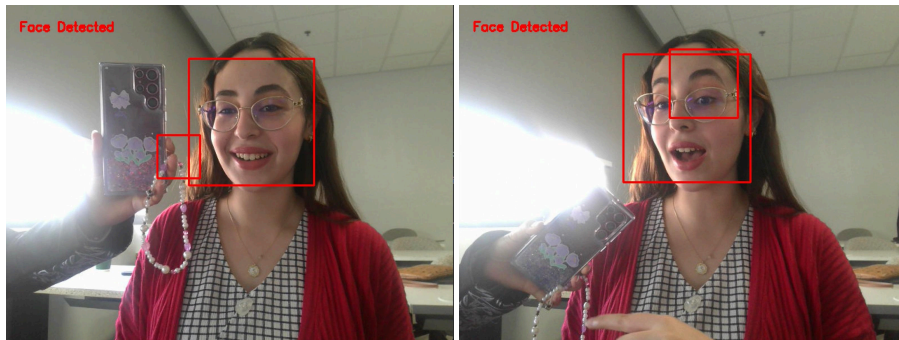


Figure 4: Face Detection Errors

It is still crucial to note, however, that holding a cell phone close to the face—especially near the cheek or jawline—is a significant issue that we identified in our trials (see Figure 4). In these cases, instead of identifying the face independently, the Haar Cascade model frequently produces inaccurate bounding boxes that include tiny portions of the face and phone. This behavior implies that the model is confused by the phone's close proximity and interprets some areas as a single entity. This kind of inaccuracy reveals a

weakness in the model's capacity to distinguish between items that are very close to one another.

Overall analysis of Haar Cascade model:

- **Accuracy:** Although Haar Cascade works well for frontal face detection, it is a little less accurate than more recent object detection algorithms like CNN-based techniques, YOLO and SSD. It has trouble with changes in position and sometimes lighting and occlusions. It is also prone to increased false positive and false negative detection rates. Research indicates that, particularly in difficult situations, Haar Cascade may have less precision and recall compared to other techniques like RetinaFace. Nonetheless, some research has used the entropy parameter to obtain excellent accuracy, with Haar Cascade segmentation results reaching 98.03% [19][20].
- **Complexity:** The Haar Cascade technique is relatively simple to develop and has low processing costs compared to more complicated algorithms like CNNs. Because of this, it can be used with low-specification devices [19].
- **Speed:** Haar Cascade is renowned for its quick detection. It can operate at a good time per second because of its low compute requirements, which makes it appropriate for real-time applications like webcam-based face detection and video surveillance. When compared to MTCNN (Multi-Task CNN), YOLOFace, and RetinaFace, Haar Cascade showed the quickest inference times [19].

4.2 Face Detection Sample Results

A small evaluation script was implemented separately to test the Haar Cascade classifier on a subset of 28 labeled images from the WIDER Face dataset [21]. The script generated a confusion matrix and calculated precision, recall, F1-score, and accuracy values to evaluate the detection performance. Each image was manually labeled as either containing a face (1) or not (0). The evaluation results are shown in Figure 5, which displays the confusion matrix and performance metrics.


```

=== Face Detection Evaluation Results ===
Confusion Matrix:
[[ 1  3]
 [ 6 18]]
Precision: 0.86
Recall: 0.75
F1-Score: 0.80
Accuracy: 0.68

```

Figure 5: Confusion Matrix

As shown, the model achieved a precision of 86%, Recall of 75%, F1-Score of 80%, and an Accuracy of 68%. This means that the classifier identified 18 real faces (true positives) while missing 6 (false negatives). It also mistakenly detected faces in 3 images where none were present (false positives), and correctly rejected 1 non-face image (true negative).

These results show that the Haar Cascade is efficient in detecting clear, front-facing faces, but it struggles in complex or crowded scenes where faces are small, side-viewed, or partially hidden. Despite these limitations, its high precision makes it suitable as a fast, lightweight “gatekeeper” stage in our hybrid detection system.

4.3 Object Detection Results

For the purpose of showcasing the performance of our YOLO11-based object detection model, it would be important to highlight the comparison between our model and the pre-existing models as mentioned before. This consists of an examination, which explains the positives as well as the negative impacts of the model.

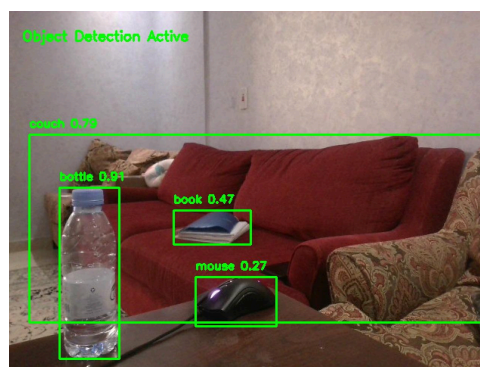


Figure 6: Object Detection Result

The YOLO11 model demonstrates its skills in object detection and localization by correctly identifying many objects in the scene. The model correctly identifies a "couch,"

"mouse," "bottle," and "book" with the corresponding confidence values, as shown in Figure 6. Each object is tightly enclosed by the bounding boxes that are produced, demonstrating precise object boundary localization. For example, the bounding box surrounding the "bottle" accurately depicts its shape and location on the table, while the bounding box placement around the "book" makes it easy to identify it from the couch. This precise identification of several different things in a single image demonstrates the model's capacity for sophisticated scene comprehension.

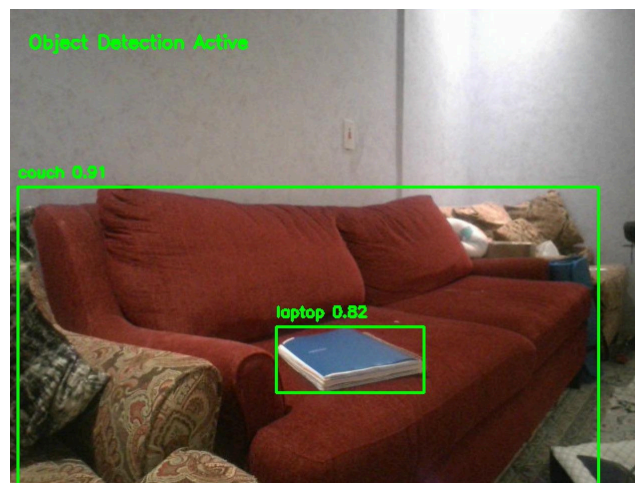


Figure 7: Object Detection Error

Despite the model performing well overall, there were a few instances of failure. A "book" laying on the couch in Figure 7 is incorrectly labeled as a "laptop" with a confidence score of 0.82, despite the "couch" being accurately identified with high confidence. In spite of the comparatively high confidence, this misidentification suggests that there may be confusion between the visual characteristics of laptops and books, especially when viewed from specific angles. The model might be focusing on the flat surface and rectangular shape shared by both objects. This draws attention to a flaw in the model's capacity to distinguish between seemingly similar items based on minute variations in size or texture. This kind of misclassification might be lessened with additional training using a more varied dataset that has examples of books in different orientations.

Overall analysis of the YOLO11 model:

- **Accuracy:** While exact performance varies depending on the model variation and dataset, YOLO11 often provides competitive accuracy when compared to other object detection models [22]. While Faster R-CNN can attain great precision, especially in complex images, YOLO11 often delivers a better mix of speed and accuracy [23]. YOLO11 usually shows better accuracy than SSD, especially in low light. YOLO11 maintains a competitive speed-to-accuracy ratio while RT-DETR, a real-time version of DETR, uses transformer topologies to achieve high accuracy [24]. Furthermore, YOLO11 is computationally efficient without sacrificing accuracy, achieving a higher mAP on the COCO dataset with 22% fewer parameters than YOLOv8 [13].
- **Complexity:** YOLO11 makes use of an enhanced backbone and neck design that improves feature extraction and occlusion handling making it of medium complexity [25] while SSD, for example, is easier to train and deploy because of its simplified architecture.
- **Speed:** In applications like autonomous driving and traffic monitoring, the model's inference speeds of up to 290 frames per second enable rapid reaction times [26]. Even faster speeds can be achieved by smaller YOLO11 variations, including YOLOv11-n, with testing demonstrating up to 170.74 FPS and 5.86 ms on GPU. However, it's crucial to note that larger models like YOLOv11-x, while excelling in accuracy, may have slower speeds [22].

Conclusion & Future Work

In this paper, using the advantages of the YOLO11 model and the Haar Cascade classifier, we introduced a face and object detection system. While YOLO11 showed its capacity to precisely identify a variety of objects inside increasingly complicated situations, the Haar Cascade classifier offered a computationally effective face identification method appropriate for real-time applications where speed is crucial. The outcomes demonstrate how complementary these strategies are. Because of its speed, the Haar Cascade can be used for preliminary face detection; YOLO11 can then be used for more in-depth object recognition inside the recognized scene. This combination enables a balance between precision and speed, meeting the needs of many applications.

The **Hybrid Model for Face Detection**, which combines Haar Cascade with more sophisticated techniques like SSDs or CNN-based methods to improve face detection

accuracy while maintaining reasonable speed, is one of several ways to improve the performance and robustness of our face and object detection system [27]. To increase overall accuracy, this may entail first detecting face regions using Haar Cascade and then refining and validating those detections using SSD. Additionally, the training dataset for both models can be improved by **data augmentation and training**, with a special emphasis on photos with difficult lighting, occlusions, and object pose variations. Fake faces can be produced by generative models to increase diversity and improve system performance in many scenarios. Finally, **adaptive feature normalization** approaches can be used to improve the models' ability to generalize to unanticipated conditions, such as partial coverings from handheld objects [28].

References

- [1] Yousif, I., Burns, L., Kalach, F. E., & Harik, R. (2024). Leveraging computer vision towards high-efficiency autonomous industrial facilities. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-024-02396-1>
- [2] Edozie, E., Shuaibu, A. N., John, U. K., & Sadiq, B. O. (2025). Comprehensive review of recent developments in visual object detection based on deep learning. *Artificial Intelligence Review*, 58(9). <https://doi.org/10.1007/s10462-025-11284-w>
- [3] Mohana, M., & Subashini, P. (2025). Analysing the performance of Viola-Jones and multi-task convolution neural networks face detection algorithms using real-time video sequences. *International Journal of Computational Vision and Robotics*, 15(3), 286–311. <https://doi.org/10.1504/ijcvr.2025.146293>
- [4] Raghavi, S., Ranjith, R., & Chandrasekar, A. (2024). Fatigue and Sluggishness Detection Using Machine Learning: A Haar Algorithmic Approach. 2024 *International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI)*, 1–5. <https://doi.org/10.1109/accai61061.2024.10601943>
- [5] Ali, M. L., & Zhang, Z. (2024). The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection. *Computers*, 13(12), 336–336. <https://doi.org/10.3390/computers13120336>
- [6] Rane, N. (2023). YOLO and Faster R-CNN object detection for smart Industry 4.0 and Industry 5.0: applications, challenges, and opportunities. *Social Science Research Network*. <https://doi.org/10.2139/ssrn.4624206>
- [7] He, P., Chen, W., Pang, L., Zhang, W., Wang, Y., Huang, W., Han, Q., Xu, X., & Qi, Y. (2024). The survey of one-stage anchor-free real-time object detection algorithms. 2–2. <https://doi.org/10.1117/12.3012931>
- [8] Gheisari, M., Ebrahimzadeh, F., Rahimi, M., Moazzamigodardi, M., Liu, Y., Dutta Pramanik, P. K., Heravi, M. A., Mehbodniya, A., Ghaderzadeh, M., Feylizadeh, M. R., & Kosari, S. (2023). Deep learning: Applications, architectures, models, tools, and frameworks: A comprehensive survey. *CAAI Transactions on Intelligence Technology*. <https://doi.org/10.1049/cit2.12180>
- [9] Pensar, H., & Rousi, R. (2023). The resources to balance – Exploring remote employees’ work-life balance through the lens of conservation of resources. *Cogent Business & Management*, 10(2). <https://doi.org/10.1080/23311975.2023.2232592>
- [10] Alsuwaylimi, A. A., Alanazi, R., Alanazi, S. M., Alenezi, S. M., Saidani, T., & Ghodhban, R. (2024). Improved and Efficient Object Detection Algorithm based on YOLOv5. *Engineering, Technology & Applied Science Research*, 14(3), 14380–14386. <https://doi.org/10.48084/etasr.7386>
- [11] Du, H., Shi, H., Zeng, D., Zhang, X.-P., & Mei, T. (2022). The Elements of End-to-end Deep Face Recognition: A Survey of Recent Advances. *ACM Computing Surveys*. <https://doi.org/10.1145/3507902>
- [12] Haruna, Y., Qin, S., Adama Chukkol, A. H., Yusuf, A. A., Bello, I., & Lawan, A. (2025). Exploring the synergies of hybrid convolutional neural network and Vision Transformer architectures for computer vision: A survey. *Engineering Applications of Artificial Intelligence*, 144, 110057. <https://doi.org/10.1016/j.engappai.2025.110057>
- [13] Ultralytics. (2025, November 23). Ultralytics YOLO11. Ultralytics YOLO Docs. <https://docs.ultralytics.com/models/yolo11/#overview>

- [14] OpenCV: Cascade Classifier. (n.d.). https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- [15] YOLO11 Breakdown: Architecture and Benchmarks. Roboflow Blog. Retrieved from <https://blog.roboflow.com/yolo11-breakdown/>
- [16] S4m. (n.d.). *GitHub - s4m520/Face-and-Object-Detection*. GitHub. <https://github.com/s4m520/Face-and-Object-Detection.git>
- [17] Google Colab. (n.d.). <https://colab.research.google.com/drive/18YCgcIOAGSJdWyr9L1CXZSpLAWXLF6qp?authuser=2#scrollTo=14rtGDIGw4wi>
- [18] Evaluating Object Detection Models: Guide to Performance Metrics. (2019, October 5). Manal El Aidouni. <https://manalaidouni.github.io/Evaluating-Object-Detection-Models-Guide-to-PerformanceMetrics.html>
- [19] *Comparative analysis of Multi-Face detection methods in classroom environments: Haar Cascade, MTCNN, YOLOFace, and RetinaFace*. (2024, October 30). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/10823781>
- [20] Andrean, M., Shidik, G., Naufal, M., Zami, F., Winarno, S., Azies, H., & Putra, P. (2024). Comparing Haar Cascade and YOLOFACE for Region of Interest Classification in Drowsiness Detection. *JURNAL MEDIA INFORMATIKA BUDIDARMA*. https://www.researchgate.net/publication/382629349_Comparing_Haar_Cascade_and_YOLOFACE_for_Region_of_Interest_Classification_in_Drowsiness_Detection
- [21] P. Gupta, "WIDER_FACE_A_Face_Detection_Dataset," Kaggle, <https://www.kaggle.com/datasets/iamprateek/wider-face-a-face-detection-dataset/data>
- [22] Kishor, R. (2024). Performance Benchmarking of YOLOV11 variants for Real-Time Delivery Vehicle Detection: A Study on accuracy, speed, and computational trade-offs. *Asian Journal of Research in Computer Science*, 17(12), 108–122. <https://doi.org/10.9734/ajrcos/2024/v17i12532>
- [23] *YOLO vs Faster RCNN for Object Detection and Recognition | International Research Journal of Multidisciplinary Scope (IRJMS)*. (2025, August 5). International Research Journal of Multidisciplinary Scope (IRJMS) | <https://www.irjms.com/journal/yolo-vs-faster-rcnn-for-object-detection-and-recognition/>
- [24] Ultralytics. (2025, October 25). *YOLO11 vs RTDETRv2: A Technical Comparison of Real-Time Detectors*. <https://docs.ultralytics.com/compare/yolo11-vs-rtdestr/>
- [25] Ghosh, A., & Ghosh, A. (2025, March 10). *YOLO11: Redefining Real-Time Object Detection - Tutorial*. LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow With Code, & Tutorials. <https://learnopencv.com/yolo11/>
- [26] Alif, M. a. R. (2024, October 30). *YOLOV11 for Vehicle Detection: Advancements, performance, and applications in intelligent transportation systems*. arXiv.org. <https://arxiv.org/abs/2410.22898>
- [27] *A hybrid model for face detection using HAAR cascade classifier and single shot Multi-Box detectors based on open CV | International Research Journal of Multidisciplinary Scope (IRJMS)*. (2024, February 3). International Research Journal of Multidisciplinary Scope (IRJMS) | <https://www.irjms.com/journal/a-hybrid-model-for-face-detection-using-haar-cascade-classifier-and-single-shot-multi-box-detectors-based-on-open-cv/>
- [28] Wang, Z. K. (2025). Current developments and future trends in face recognition technology based on artificial

intelligence. *Proceedings, International Conference on Remote Sensing, Mapping, and Image Processing*, 13650.
<https://www.spiedigitallibrary.org/conference-proceedings-of-spie/13650/3067564/Current-developments-and-future-trends-in-face-recognition-technology-based/10.1117/12.3067564.short>